Fine-tuning per un solo compito: "il documento supporta il claim?" — panoramica

Questa cartella è la variante **specializzata** di ../fine_tuning/. Lì l'obiettivo era un fact checker che restasse anche un *decision model generale* (domande `choice`, `noul`, `score` qualsiasi). Qui il compito è **uno solo**:

> dato un documento e un claim, dire se il claim è vero (supportato dal documento) o falso (contraddetto o non stabilito).

Il documento è sempre fornito, non c'è retrieval. Le capacità generali del modello di partenza non interessano più: questo cambia la risposta a quasi tutte le domande di progetto (architettura, replay, varianti di domanda, loss, valutazione).

1. La domanda e la risposta in breve

Domanda: con un compito solo binario conviene ancora fare fine-tuning di un decision model, o è meglio costruire un classificatore come FactCG e MiniCheck? E se si parte da un LLM, conviene cambiargli la testa?

Risposta (dettagli e numeri in 00a_scelta_approccio.md):

1. **FactCG e MiniCheck non sono un'architettura, sono una ricetta di dati.** Entrambi prendono un backbone esistente (DeBERTa, RoBERTa, Flan-T5) e lo addestrano su dati sintetici costruiti bene. La stessa ricetta applicata a un LLM da 7B (Bespoke-MiniCheck-7B) passa da 63,6 a 77,4 su LLM-AggreFact. Quindi "decision model o FactCG" è una falsa alternativa: **si prende la ricetta di dati di FactCG/MiniCheck e la si applica al backbone migliore**.
2. **Il backbone migliore oggi è un decision model usato con una sola domanda `noul`.** Winnow-12B e JPT-9B con la sola `support_simple` fanno già 76,7 e 76,2 zero-shot, più di FactCG-DeBERTa (75,6) e MiniCheck-FT5 (75,0) dopo il loro training.
3. **Non cambiare la testa.** Leggere i logit delle due lettere della risposta *è già* una testa di classificazione binaria (un vettore lineare sullo stato nascosto), inizializzata con quello che il modello sa. Sostituirla con una testa nuova casuale dà la stessa capacità ma butta via il punto di partenza. La testa si "estrae" solo alla fine, per servire il modello come classificatore (Step 8).
4. **Un classificatore piccolo stile FactCG serve solo se il costo d'inferenza conta molto** (milioni di claim, CPU). In quel caso il modo migliore di ottenerlo è **distillarlo** dal decision model fine-tunato, non addestrarlo da zero sugli stessi dati.

2. Da dove partiamo

LLM-AggreFact test (29.320 righe, 11 fonti), BAcc macro con soglia fittata sul dev di AggreFact (numeri "dev-tuned", secondari: vedi 00b). La riga `support_simple` è calcolata in questa sessione dalle cache esistenti, senza nuova inferenza.

Modello	Regola	Parametri	BAcc macro	AUC macro	Soglia
Winnow-12B-BF16	3 domande (`min`)	12B	77,42	85,02	0,22
Winnow-12B-BF16	**solo `support_simple`**	12B	**76,69**	**84,03**	0,95
JPT-9B	3 domande	9B	76,71	84,57	0,72
JPT-9B	**solo `support_simple`**	9B	**76,20**	**84,07**	0,90
Bespoke-MiniCheck-7B (leaderboard)	1 domanda Yes/No	7B	77,4		

Modello	Regola	Parametri	BAcc macro	AUC macro	Soglia
FactCG-DeBERTa-v3-L (rimisurato qui)	classificatore	0,4B	75,64		0,5
MiniCheck-Flan-T5-L (leaderboard)	classificatore	0,8B	75,0		0,5
InternLM2.5-7B-chat (base di Bespoke, prima del training)		7B	63,6		

Due letture: - la domanda singola costa ~0,5–0,7 punti zero-shot ma è **3 volte più economica** in inferenza; - le soglie ottimali 0,90–0,95 dicono che zero-shot la domanda semplice è **sbilanciata verso "sì"**: è esattamente il tipo di difetto che il fine-tuning corregge con poco.

3. Il percorso in 10 step

Stessa struttura di ../fine_tuning/, con gli step semplificati dove il compito unico lo permette.

```
0.  Split e protocollo     train / dev / test nostri, congelati; AggreFact solo test
0a. Scelta dell'approccio  decision model + 1 domanda (principale); encoder stile FactCG (riferimento / dist
      |
1.  Baseline               decision model (1 e 3 domande), modelli base, FactCG, MiniCheck
      |
2.  Dataset                2a fonti esistenti
      |                    2b generazione stile MiniCheck (C2D/D2C) e FactCG (CG2C)
      |                    2c replay: non serve (perché)
      |                    2d varianti di domanda: poche, opzionali
      |                    2e mix in due stadi (come MiniCheck/FactCG)
      |
3.  Rendering              una sola domanda, prompt nativo, chunking
4.  Loss                   CE binaria sul margine fra le due lettere
5.  Training               LoRA          <----------┐
6.  Valutazione            suite senza oblio        |
7.  Esperimenti            una variazione alla volta
8.  Merge / export         pesi fusi, testa estratta, (opz.) distillazione in encoder
```

Step	File	Cosa cambia rispetto a `fine_tuning/`
0. Split	00b_split_e_protocollo.md	stesso protocollo; sparisce il blocco "oblio"
0a. Approccio	00a_scelta_approccio.md	**nuovo**: decision model vs FactCG/MiniCheck, testa nuova vs lettere
1. Baseline	01_baseline.md	baseline a una domanda; FactCG e MiniCheck nella stessa suite
2a. Fonti	02a_dataset_fonti_esistenti.md	stesse fonti; ANLI "difficile" scelto con il nostro modello
2b. Generazione	02b_dataset_generazione_llm.md	C2D/D2C e CG2C al centro, non come aggiunta

Step	File	Cosa cambia rispetto a `fine_tuning/`
2c. Replay	02c_replay_e_forgetting.md	**eliminato**; resta un ancoraggio opzionale per la calibrazione
2d. Varianti	02d_varianti_domanda.md	da ~40 varianti a un prompt fisso (+ poche varianti opzionali)
2e. Mix	02e_mix_finale.md	curriculum a due stadi come MiniCheck/FactCG
3. Rendering	03_rendering_formato.md	solo S1; chunking come MiniCheck
4. Loss	04_loss.md	niente KL di replay; equivalenza lettere = testa binaria
5. Training	05_training.md	learning rate più libero; traccia encoder in appendice
6. Valutazione	06_valutazione.md	niente oblio; più peso a calibrazione, costo, documenti lunghi
7. Esperimenti	07_esperimenti.md	nuova fase A (architettura) e S (distillazione)
8. Export	08_merge_export.md	testa binaria estratta; distillazione opzionale
Rischi	09_rischi_checklist.md	rischi specifici del modello specializzato

4. Le decisioni chiave

1. **Backbone: decision model, una domanda `noul`.** Winnow-12B come candidato finale, JPT-9B per iterare (più piccolo, gira sul portatile in QLoRA).
2. **Dati: ricetta MiniCheck/FactCG, estesa.** C2D/D2C (rilasciati), CG2C (da rigenerare), ANLI difficile, più le fonti già preparate in `data_prep/`.
3. **Loss solo sulle due lettere.** È una regressione logistica sul margine `z_vero − z_falso`: la stessa cosa che fa la testa binaria di FactCG, con un'inizializzazione migliore.
4. **Niente replay obbligatorio.** Il modello può dimenticare le altre domande: non servono. Si controlla solo la calibrazione.
5. **Prompt fisso.** Chi chiama il modello è il nostro codice, sempre con la stessa domanda: le varianti diventano un esperimento, non un requisito.
6. **Due stadi.** Prima dati "logici" (NLI difficile, C2D, multi-hop), poi dati realistici (D2C, riassunti, CG2C-Doc), come nei due paper.
7. **Controlli obbligatori.** Lo stesso training su un LLM base (Gemma-4-12B, Qwen3.5-9B) e un encoder stile FactCG sugli stessi dati: dicono quanto vale l'inizializzazione e quanto la taglia.
8. **Protocollo invariato.** Scelte sul nostro dev; LLM-AggreFact solo come test zero-shot.

5. Obiettivi misurabili

Obiettivo	Soglia
LLM-AggreFact zero-shot, BAcc macro, **una sola domanda**	$> 77{,}4$ (Bespoke-MiniCheck-7B) con margine; puntare a $\geq 78{,}5$
AUC macro su AggreFact	$> 85{,}0$ (Winnow a 3 domande): il guadagno deve venire dall'ordinamento, non dalla soglia
Soglia ottimale sul nostro dev	fra 0,3 e 0,7 (oggi 0,90–0,95 con `support_simple`)
ECE sul nostro dev	$\leq$ baseline
test_ood	nessun peggioramento rispetto alla baseline
Test comportamentali	"solo claim" $\ll 0{,}5$; rimozione dell'evidenza fa scendere la probabilità in $> 80\%$ dei casi
Costo	1 forward per (chunk, claim), 1 solo token letto

6. Tempi e risorse (stime)

Fase	Macchina	Tempo indicativo
Split, baseline dalle cache	qualsiasi	ore (gran parte già calcolata)
Generazione C2D/D2C/CG2C	L40S con modello locale, o API	1–2 giorni per ~20k esempi verificati
Pilota JPT (1 domanda)	portatile 24 GB (QLoRA) o L40S	3–6 h per corsa (un terzo degli esempi rispetto a S3)
Winnow	L40S	4–12 h per corsa
Encoder stile FactCG (controllo / studente)	portatile 24 GB	1–4 h per corsa

7. Relazione con gli altri documenti

- ../fine_tuning/: piano per il fact checker che resta decision model generale. Vale ancora se in futuro servono anche altre domande sullo stesso modello.
- ../models/FactCG.md, ../models/minicheck.md: analisi dei due paper, da cui vengono ricetta di dati, chunking e curriculum.
- ../prompt_formats.md: prompt esatti dei modelli.